Trapezoid: A Versatile Accelerator for Dense and Sparse Matrix Multiplications

August 12, 2026

Matrix Multiplication

Consider

$$\mathbf{A} \in \mathbb{R}^{M \times K}, \qquad \mathbf{B} \in \mathbb{R}^{K \times N}, \qquad \mathbf{C} = \mathbf{AB}.$$

Each output element is

$$\boxed{C_{mn} = \sum_{k=0}^{K-1} A_{mk} B_{kn}}$$

The computation contains three indices:

$$\boxed{m \rightarrow \text{row of } A \qquad k \rightarrow \text{reduction dimension} \qquad n \rightarrow \text{column of } B}$$

Architectural question

The question is how can we leverage the sparsity in Hardware and data flows, to optimize the computation?

Sparsity

For sparse matrices, many elements satisfy

$$A_{mk} = 0 \qquad \text{or} \qquad B_{kn} = 0.$$

A dense matrix multiplication still attempts to perform

$$A_{mk} B_{kn}$$

even when the result is guaranteed to be zero.

Dense

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Nearly every multiply is useful.

Sparse

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 9 \end{bmatrix}$$

Most intersections are useless.

Fundamental Matrix-Multiplication Dataflows

The same equation

$$C_{mn} = \sum_k A_{mk} B_{kn}$$

can be organized in different ways.

Inner Product (IP)

$$C_{mn} = \sum_k A_{mk} B_{kn}$$

```
for m
  for n
    for k
      C_mn += A_mk B_kn
```

One output element at a time

Outer Product (OP)

$$C = \sum_k A_{:,k} B_{k,:}$$

```
for k
  for m
    for n
      C_mn += A_mk B_kn
```

Complete Matrix update at a time

Gustavson

$$C_{m,:} = \sum_k A_{mk} B_{k,:}$$

```
for m
  for each nonzero A_mk
    for n
      C_mn += A_mk B_kn
```

One output row at a time

Why Dataflow Matters for Sparse Matrices

IP

$A_{m,:} \times B_{:,n}$

- Simple Multiply and Accumulate
- Excellent for dense matrices
- Many useless intersections when sparse

OP

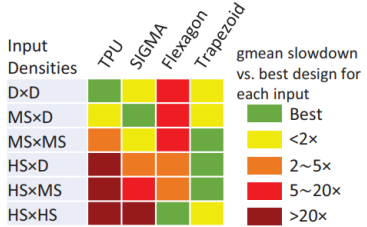
$A_{:,k} \times B_{k,:}$

- Naturally exploits sparsity
- Generates many partial products
- Requires storage of entire matrix at a time

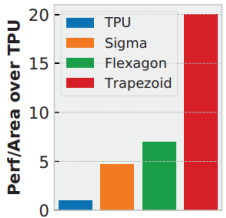
Gustavson

$A_{mk} B_{k,:}$

- Skips zero entries of A
- Avoids many useless intersections
- Requires irregular gather accesses to B

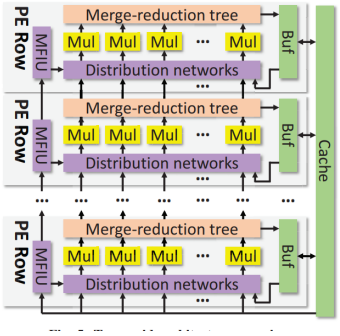


(a) Per workload.



(b) Gmean.

Trapezoid: Architectural Overview



- Extends a 128×128 2-D spatial array.
- Retains the standard dense IP dataflow.
- Adds sparse-aware IP and Gustavson-based dataflows.
- Uses a multi-level memory hierarchy for irregular accesses.

Trapezoid: Adaptive Dataflows

Input sparsity	Dataflow	Purpose
$D \times D$	IP	Dense computation
$MS \times D$	TrIP	Sparse A
$MS \times MS$	TrIP	Sparse intersection
$HS \times D$	TrGS	Spatial Gustavson for Highly Sparse A
$HS \times MS$	TrGS	Spatial Gustavson for Highly Sparse A
$HS \times HS$	TrGT	Temporal Gustavson for Highly Sparse A amd B

Main idea

The hardware does not use one fixed dataflow. It chooses the dataflow according to the sparsity characteristics.

TrIP: Multi-Fiber Inner Product

Standard IP evaluates one fiber intersection:

$$A_{m,:} \cdot B_{:,n}.$$

TrIP processes multiple fibers simultaneously.

$$\boxed{4 \text{ rows of } A \times 4 \text{ columns of } B = 16 \text{ possible intersections}}$$

Key benefit

Instead of serially determining one intersection at a time, TrIP exposes up to 16 fiber intersections concurrently.

Multi-Fiber Intersection Unit (MFIU)

The MFIU identifies useful intersections before multiplication.

Suppose two fibers have occupancy masks

$$A_{\text{mask}} = [1, 0, 1, 0]$$

and

$$B_{\text{mask}} = [1, 0, 1, 1].$$

Their intersection is

$$\boxed{A_{\text{mask}} \wedge B_{\text{mask}} = [1, 0, 1, 0]}$$

Therefore useful coordinates are

$$k = \{0, 2\}.$$

Bitmask intersection $\rightarrow$ Prefix sum $\rightarrow$ Index shifting/routing

TrIP Workflow: From Sparse Fibers to Output

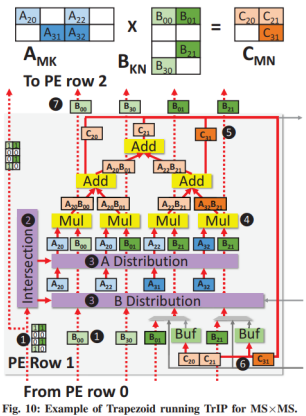


Fig. 10: Example of Trapezoid running TrIP for MS×MS.

1. Intersection — Find common nonzero positions of the A and B fibers.

2. B Distribution — Route the required B_{kn} values to the corresponding PEs.

3. A Distribution — Route the matching A_{mk} values to the same PEs.

4. Multiply — Each PE computes a useful product: $A_{mk}B_{kn}$.

5. Accumulate — Partial products contributing to the same C_{mn} are added.

6. Buffer / Forward — Completed C values are buffered, while required B values move to the next PE row.

Key idea

Find useful intersections → route operands → multiply and accumulate

TrIP PE-Row Microarchitecture

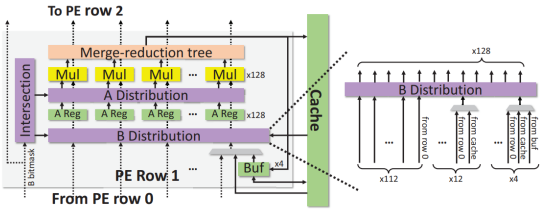


Fig. 6: PE row microarchitecture.

Each PE row contains:

$$\text{Local Buffer} + \text{MFIU} + \text{Distribution Networks} + \text{Merge-Reduction Tree}$$

- MFIU determines useful intersections.
- Distribution networks route corresponding A and B values.
- Multipliers compute useful products.
- Reduction tree accumulates partial products.

Why TrIP Is Not Enough

At high sparsity, even sparse-aware IP becomes inefficient.
For Gustavson:

$$\boxed{C_{m,:} = \sum_{k:A_{mk}\neq 0} A_{mk} B_{k,:}}$$

If

$$A_{m,:} = [2, 0, 5, 0],$$

then

$$C_{m,:} = 2B_{0,:} + 5B_{2,:}.$$

Only rows $B_{0,:}$ and $B_{2,:}$ need to be accessed.

Key idea

Gustavson directly follows the nonzeros of A and gathers the corresponding rows of B.

TrGT: Temporal Gustavson

TrGT processes sparse rows of B **temporally**.
For

$$C_{m,:} = \sum_{k:A_{mk}\neq 0} A_{mk} B_{k,:},$$

the accelerator:

1. Finds a nonzero A_{mk}.
2. Fetches row $B_{k,:}$.
3. Broadcasts A_{mk}.
4. Multiplies $A_{mk} B_{k,n}$.
5. Accumulates into $C_{m,n}$.

TrGS: Spatial Gustavson

TrGS exploits the fact that the N dimension can be treated as a dense dimension when B is dense or mildly sparse.

For one row of B:

$$B_{k,:} = [B_{k0}, B_{k1}, \ldots, B_{k,N-1}],$$

different elements are mapped spatially to different multipliers.

A_{mk} is broadcast $\quad \rightarrow \quad$ $B_{k,n}$ values are routed spatially

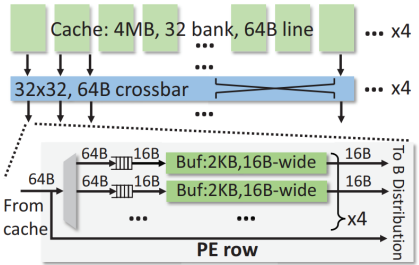
Memory Hierarchy: Handling Irregular Accesses

Gustavson introduces irregular gather accesses to B.
A naive implementation would repeatedly access:

$$\text{HBM} \rightarrow B_{k,:}$$

which can make memory bandwidth the bottleneck.
Trapezoid introduces:

$$\boxed{\text{HBM} \rightarrow \text{Global Cache} \rightarrow \text{Row-local Buffer} \rightarrow \text{PEs}}$$



Putting the Architecture Together

Sparsity regime	Dataflow	Main mechanism
Dense	IP	2-D spatial array
Mild sparsity	TrIP	MFIU + routing
High A, dense/mild B	TrGS	Spatial Gustavson
High A, high B	TrGT	Temporal Gustavson + cache

Dataflow selection $\approx f(\text{sparsity}, \text{arithmetic intensity}, \text{memory traffic})$

Core architectural contribution

Trapezoid combines multiple specialized execution strategies inside a common spatial-array architecture.

Roofline Perspective

The achievable performance is bounded by

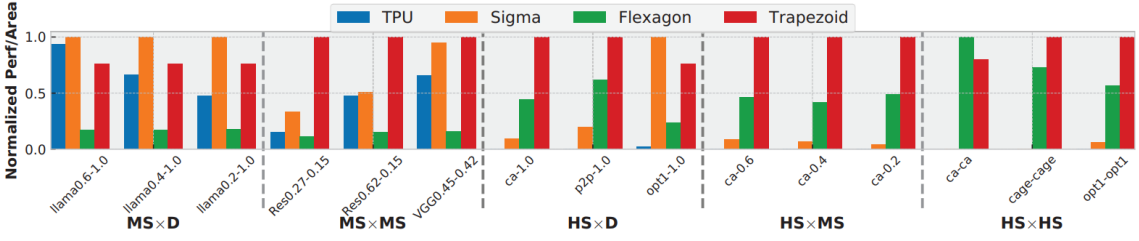
$$\boxed{P = \min\left(P_{\text{compute}}, BW \times AI\right)}$$

where

$$AI = \frac{\text{operations}}{\text{bytes transferred}}.$$

- High AI: compute throughput dominates.
- Low AI: memory bandwidth dominates.
- Sparsity generally reduces useful computation and changes the memory/computation balance.

Performance Across the Sparsity Spectrum



Why Trapezoid Performs Well

Mildly sparse workloads

MFIU $\rightarrow$ many intersections in parallel

For example, TrIP can examine

$$4 \times 4 = 16$$

fiber intersections simultaneously.

Highly sparse workloads

Gustavson $\rightarrow$ avoid ineffectual intersections

and

Memory hierarchy $\rightarrow$ reduce irregular-memory traffic

Configuration and Area of Baseline Systems

Component	**Configuration**	**Area (mm^2)**
TPUv3-like	1 GHz, 128 × 128 MACs, 16 MB SRAM, 2 TB/s HBM	41.0
SIGMA	1 GHz, 128 × 128 MACs, 17 MB SRAM, 2 TB/s HBM	62.3
Scaled-up Flexagon	1 GHz, 67 Flexagon instances, 67 × 64 MACs, 67 MB SRAM, 2 TB/s HBM	80.8
Trapezoid	1 GHz, 128 × 128 MACs, 17 MB SRAM, 2 TB/s HBM	81.9

Energy and Memory-Traffic Perspective

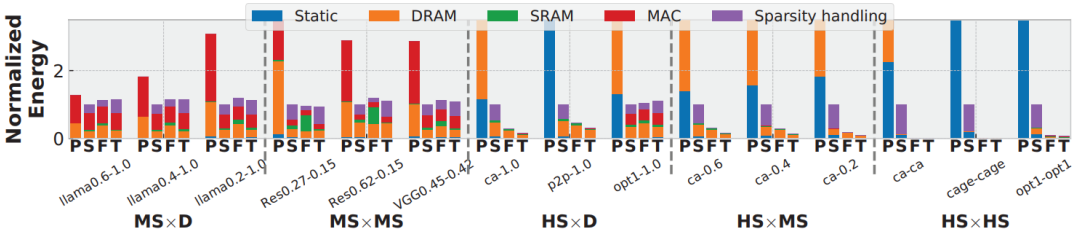


Fig. 26: Energy breakdown comparison on 15 workloads (normalized to SIGMA). P: TPU, S: SIGMA, F: Flexagon, T: Trapezoid.

Overall Perspective

1. **Dataflow becomes an architectural parameter**

$$\boxed{\text{Same } C = AB \qquad \text{different execution strategy}}$$

2. **Sparse acceleration is more than zero skipping**

$$\boxed{\text{Intersection detection} + \text{routing} + \text{reduction}}$$

3. **Algorithm and memory hierarchy are coupled**

$$\boxed{\text{Gustavson} \rightarrow \text{irregular access} \rightarrow \text{cache/buffer co-design}}$$

4. **The optimal architecture depends on arithmetic intensity**

$$AI \uparrow \Rightarrow \text{compute-oriented TrIP}$$

$$AI \downarrow \Rightarrow \text{Gustavson-based execution}$$